



Trajecto

Équipe 10

Étudiants :

Benjamin Blanc
Manon Duquesne
Hugo Lhommeau
Jérémy L'Hostis
Alexandre Yu

Encadrant :

Kévin Leroy

Table des matières

Introduction	1
1 Planning	2
1.1 Phase d'analyse	2
1.2 Phase de développement	2
2 Fonctionnalités principales	3
2.1 Gestion des comptes utilisateur	3
2.2 Gestion des lignes d'exploitation	4
2.3 Gestion des trajets proposés	4
2.4 Recherche d'un trajet	5
2.5 Positionnement commercial	6
3 Modélisation	6
3.1 Architecture générale	6
3.2 CLASSE OBJET :	7
3.3 Diagrammes des cas d'utilisation	7
3.4 Diagrammes d'activité	7
3.5 Diagramme de classes	7
3.6 Modèle de base de données	7
Conclusion intermédiaire	8

Introduction

Et si chacune et chacun pouvait imaginer sa propre manière de voyager en train ? C'est autour de cette idée que s'articule **Trajecto**, une plateforme destinée à proposer, gérer et rechercher des trajets ferroviaires en s'appuyant sur les données du réseau SNCF.

L'objectif de notre application est de fournir une plateforme permettant à une entreprise ferroviaire de gérer son offre de transport et à ses clients de rechercher facilement les trajets disponibles. L'interface graphique étant prise en charge par les webmasters de l'entreprise, notre travail se concentre sur la conception et le développement d'une API robuste, sécurisée et exploitable par différents clients.

L'application s'articule principalement autour de quatre fonctionnalités : de la gestion des comptes utilisateurs, à trois niveaux d'habilitation différents, à la recherche de trajets précis, en passant par l'organisation des lignes d'exploitation.

En complément de ces fonctionnalités essentielles, nous étudierons un positionnement commercial propre à notre entreprise afin de proposer une expérience différente de celle du réseau ferroviaire traditionnel. Cette fonctionnalité pourra notamment s'appuyer sur une meilleure transparence des informations proposées au client, par exemple en détaillant la composition du prix d'un trajet.

Ce rapport d'analyse intermédiaire présente ainsi les choix effectués pour répondre à ces besoins. Nous y décrirons le fonctionnement général de l'application, les différents acteurs et leurs interactions avec l'API, la structure des données ainsi que les principaux choix d'implémentation envisagés. L'objectif est de disposer, à l'issue de cette phase, d'une vision suffisamment précise du système pour pouvoir passer au développement de l'application dans de bonnes conditions.

1 Planning

1.1 Phase d'analyse

1.2 Phase de développement

2 Fonctionnalités principales

2.1 Gestion des comptes utilisateur

2.1.a Description

Permettre aux utilisateurs de créer et gérer leur compte. Chaque compte possède un nom d'utilisateur, un mot de passe et un niveau d'habilitation parmi CLIENT, COLLABORATEUR et ADMIN. Les comptes ADMIN peuvent modifier le niveau d'habilitation des autres utilisateurs, à l'exception des autres comptes ADMIN.

2.1.b Fonctionnement

Lors de la création d'un compte, l'utilisateur renseigne un nom d'utilisateur et un mot de passe. Le compte est associé à un niveau d'habilitation CLIENT.

Un compte ADMIN peut consulter les comptes utilisateurs et augmenter ou diminuer leur niveau d'habilitation. Un ADMIN ne peut cependant pas modifier le niveau d'un autre ADMIN.

Un compte est associé à un identifiant unique, qui est utilisé pour identifier le compte dans la base de données.

Un ADMIN ne peut pas modifier le niveau d'habilitation d'un autre ADMIN.

Un utilisateur non ADMIN ne peut pas modifier les habilitations des autres comptes.

(Le mot de passe doit être stocké de manière sécurisée et ne doit jamais être enregistré en clair.)

2.1.c Prérequis

- Nom d'utilisateur
- Mot de passe
- Niveau d'habilitation lors de la création ou de la modification
- Identifiant du compte à modifier pour un ADMIN

2.1.d Tâches effectuées

- Création d'un nouveau compte utilisateur
- Connexion à un compte existant
- Modification du niveau d'habilitation d'un utilisateur
- Affichage d'un message de confirmation ou d'erreur en cas d'échec

2.1.e Exemple

Un utilisateur crée un compte avec le nom alice et un mot de passe. Son compte est créé avec le rôle CLIENT. Un ADMIN peut ensuite faire évoluer son rôle en COLLABORATEUR. L'ADMIN ne peut toutefois pas modifier le rôle d'un autre compte ayant déjà le niveau ADMIN.

2.2 Gestion des lignes d'exploitation

2.2.a Description

Permettre à un compte COLLABORATEUR de créer, modifier et supprimer des lignes d'exploitation reliant une gare de départ à une gare de terminus. Les gares utilisées sont récupérées à partir de l'API SNCF afin de garantir leur existence.

2.2.b Fonctionnement

Lors de la création ou de la modification d'une ligne, le collaborateur recherche les gares grâce à une fonctionnalité d'autocomplétion. Les résultats proposés proviennent de l'API SNCF.

Le collaborateur sélectionne une gare de départ et une gare de terminus parmi les gares existantes, puis enregistre la ligne d'exploitation.

Les deux gares doivent exister dans les données fournies par l'API SNCF.

Une ligne ne peut pas être créée si la gare de départ et la gare de terminus sont identiques.

Un compte CLIENT ne peut pas gérer les lignes d'exploitation, sauf évolution ultérieure des droits.

La suppression d'une ligne peut être refusée si des trajets sont déjà planifiés sur cette ligne, afin de préserver l'intégrité des données.

2.2.c Prérequis

- Gare de départ
- Gare de terminus
- Compte utilisateur disposant du rôle COLLABORATEUR

2.2.d Tâches effectuées

- Création d'une nouvelle ligne d'exploitation
- Modification d'une ligne existante
- Suppression d'une ligne
- Affichage d'un message de confirmation ou d'erreur en cas d'échec

2.2.e Exemple

Un collaborateur saisit "Rennes" dans le champs de départ. L'application propose notamment "Rennes" grâce à l'API SNCF. Il sélectionne Rennes comme gare de départ et Paris comme gare de terminus, puis crée la ligne Rennes → Paris.

2.3 Gestion des trajets proposés

2.3.a Description

Permettre à un compte COLLABORATEUR de planifier des trajets sur une ligne d'exploitation existante. Un trajet est défini notamment par une date et une heure de départ, un nombre de places disponibles et un tarif. Le temps de trajet est automatiquement déterminé à partir de l'API SNCF.

2.3.b Fonctionnement

Le collaborateur sélectionne une ligne d'exploitation existante, puis renseigne la date et l'heure de départ, le nombre de places et le tarif.

L'application interroge l'API SNCF afin de vérifier qu'un trajet en train est possible entre la gare de départ et le terminus. Elle récupère également les informations nécessaires au calcul automatique du temps de trajet.

Le collaborateur peut ensuite modifier ou supprimer le trajet planifié.

La ligne d'exploitation doit exister avant de pouvoir planifier un trajet.

Le trajet doit être réalisable en train selon les informations fournies par l'API SNCF.

Le temps de trajet ne doit pas être saisi manuellement par le collaborateur.

La date et l'heure de départ doivent être valides.

Le nombre de places doit être supérieur à zéro.

Le tarif doit être positif ou nul selon les règles commerciales retenues.

Un trajet ne peut pas être planifié si l'API SNCF ne permet pas de vérifier sa faisabilité.

2.3.c Prérequis

- Ligne d'exploitation
- Date de départ
- Heure de départ
- Nombre de places
- Tarif
- Compte utilisateur disposant du rôle COLLABORATEUR

2.3.d Tâches effectuées

- Création d'un trajet planifié
- Temps de trajet calculé automatiquement
- Modification ou suppression d'un trajet
- Confirmation ou message d'erreur

2.3.e Exemple

Un collaborateur sélectionne la ligne Rennes → Paris, choisit le 15 octobre à 08h00, indique 50 places disponibles et un tarif de 45 €. L'API SNCF confirme qu'un trajet est possible et permet à l'application de calculer automatiquement sa durée.

2.4 Recherche d'un trajet

2.4.a Description

Permettre à un compte CLIENT de rechercher les trajets proposés entre une gare de départ et une gare d'arrivée. Une autocomplétion facilite la sélection des gares.

2.4.b Fonctionnement

Le client saisit une gare de départ et une gare d'arrivée. L'application propose automatiquement des gares correspondant à la saisie grâce aux données issues de l'API SNCF.

Après validation de la recherche, l'application affiche les trajets proposés correspondant aux critères renseignés.

Pour chaque trajet, le client peut consulter la gare de départ, la gare d'arrivée, la date et l'heure de départ ainsi que le temps de trajet.

Les gares doivent correspondre à des gares existantes.

La gare de départ et la gare d'arrivée doivent être différentes.

Si aucun trajet ne correspond à la recherche, un message explicite est affiché.

Une recherche peut retourner plusieurs trajets pour une même liaison.

2.4.c Prérequis

- Gare de départ
- Gare d'arrivée
- Compte utilisateur disposant du rôle CLIENT

2.4.d Tâches effectuées

Renvoie une liste de trajets correspondant aux gares recherchées, comprenant au minimum : - Gare de départ - Gare d'arrivée - Date de départ - Heure de départ - Temps de trajet

2.4.e Exemple

Un client sélectionne Rennes comme gare de départ et Paris comme gare d'arrivée. L'application affiche les trajets disponibles entre ces deux gares avec leur date, leur heure de départ et leur durée.

2.5 Positionnement commercial

2.5.a Description

Proposer une fonctionnalité permettant de différencier le service du réseau commercial de la SNCF en offrant au client une tarification transparente. Cette fonctionnalité ne sera implémentée que si le temps le permet: nous n'allons pas l'explorer dans ce rapport pour l'heure.

3 Modélisation

3.1 Architecture générale

3.1.a Base de données :

Pour stocker les données, nous allons utiliser une base de données postgresQL, sur l'environnement du datalab de l'ENSAI. La base de données contiendra les informations utilisateurs, ainsi que les gares, les lignes de train et les trajets prévus. Ces données seront saisies petit à petit. Ainsi, les collaborateurs

écriront dans la base de données lorsqu'ils créeront un nouveau trajet, ou lorsqu'une nouvelle ligne sera créée.

3.1.b API :

Nous allons aussi récupérer des données via l'API de la SNCF, Navitia. Navitia est une API qui fonctionne par requêtes HTTP GET. Une requête HTTP GET est une demande envoyée à un serveur pour récupérer des informations à partir d'une URL. En envoyant cette requête, l'API demande une clé d'authentification, qui est disponible gratuitement avec une limite de 1500 appels par jour, et dont nous avons fait la demande au préalable. L'API renvoie ensuite une réponse HTTP contenant les données demandées au format JSON. Dans le cadre de notre projet, nous utiliserons l'API pour deux principales raisons :

-Récupérer les gares existantes -Récupérer les temps de trajet existants sur une ligne.

3.1.c DAO :

Les DAO, pour Data Access Object, vont nous permettre de faire le lien entre les classes métier et la base de données. Leur rôle est de gérer l'accès aux données sans que les classes métier aient besoin de connaître le fonctionnement de la base de données. Ils permettent notamment, via des requêtes SQL, d'enregistrer les objets créés par les collaborateurs (gares, trajets, utilisateurs, etc.) dans la base de données afin de les sauvegarder. Ils permettent également de récupérer les données de la base de données et de les transformer en objets Python utilisables par le reste de l'application.

3.2 CLASSE OBJET :

3.2.a CLASSE SERVICE :

3.3 Diagrammes des cas d'utilisation

3.4 Diagrammes d'activité

3.4.a Client

3.4.b Collaborateur

3.4.c Administrateur

3.5 Diagramme de classes

3.6 Modèle de base de données

Conclusion intermédiaire